

基于Transformer的中文命名实体识别

王殿云 2022211733号 2022219107班

摘要

本次实验作业探索了基于Transformer架构的中文命名实体识别(NER)任务。通过七组对照实验，系统地分析了不同Transformer结构变体对中文NER任务性能的影响，包括模型深度、宽度、位置编码方式、归一化策略以及词汇增强机制。实验结果表明，词汇增强的LeBert架构在中文NER任务上取得了最佳性能，字符集F1值达到0.7552，较基线模型提升了约3个百分点。

1. 实验要求

本实验要求基于Transformer架构实现中文命名实体识别模型，对提供的测试数据集进行序列标注。具体要求如下：

- 以Transformer为基础结构实现NER模型
- 可以采用单独的Transformer编码器、单独的Transformer解码器或编解码器的组合架构
- 模型超参数（如层数、隐藏层维度等）可自行设定
- 字向量可以随机初始化或使用预训练词向量
- 通过识别测试集中的命名实体，为每个字符生成相应的标签（如B-PER, I-LOC等）

经统计，本实验中采用的标注集包含以下**九个标签**：

- O: 非实体
- B-PER/I-PER: 人名实体的开始/中间位置
- B-LOC/I-LOC: 地点实体的开始/中间位置
- B-ORG/I-ORG: 组织实体的开始/中间位置
- B-T/I-T: 时间实体的开始/中间位置

2. 实验原理

2.1 命名实体识别任务

命名实体识别是自然语言处理的基础任务之一，目标是从非结构化文本中识别和提取特定类别的实体。在中文NER任务中，每个字符被标注为特定标签，通常采用BIO或BIOES标注体系。本实验采用BIO标注体系，其中B表示实体的开始位置，I表示实体的中间位置，O表示非实体。

2.2 Transformer架构

Transformer是一种基于自注意力机制的神经网络架构，最早由Vaswani等人[1]提出。其核心组件包括：

1. 多头自注意力机制(Multi-Head Self-Attention):

允许模型同时关注输入序列的不同位置，计算如下：

$$\text{Attention}(Q, K, V) = \text{softmax} \left(\frac{QK^T}{\sqrt{d_k}} \right) V$$

$$\text{MultiHead}(Q, K, V) = \text{Concat}(\text{head}_1, \dots, \text{head}_h) W^O$$

其中 $\text{head}_i = \text{Attention}(QW_i^Q, KW_i^K, VW_i^V)$

2. 前馈神经网络(Feed-Forward Network):

对每个位置的表示进行非线性变换：

$$\text{FFN}(x) = \max(0, xW_1 + b_1)W_2 + b_2$$

3. 层归一化(Layer Normalization):

稳定训练过程：

$$\text{LayerNorm}(x) = \gamma \frac{x - \mu}{\sqrt{\sigma^2 + \epsilon}} + \beta$$

4. 位置编码(Positional Encoding):

为模型提供位置信息：

$$\begin{aligned} \text{PE}_{(pos, 2i)} &= \sin(pos/10000^{2i/d_{model}}) \\ \text{PE}_{(pos, 2i+1)} &= \cos(pos/10000^{2i/d_{model}}) \end{aligned}$$

2.3 旋转位置编码(RoPE)

旋转位置编码[2]是一种相对位置编码方法，通过对注意力中的查询和键向量应用旋转变换来编码位置信息：

$$\begin{aligned} q'_m &= q_m \cos(m\theta) - q_{m+1} \sin(m\theta) \\ q'_{m+1} &= q_m \sin(m\theta) + q_{m+1} \cos(m\theta) \end{aligned}$$

其中 θ 为频率参数， q_m 为向量的第 m 维。RoPE能更好地捕获相对位置信息，通常比固定的正弦位置编码表现更好。

2.4 前置层归一化与后置层归一化

在Transformer结构中，归一化层的位置有两种选择：

1. 后置归一化(Post-Norm): 原始Transformer采用的方式

$$\text{output} = \text{LayerNorm}(x + \text{Sublayer}(x))$$

2. 前置归一化(Pre-Norm): 先归一化再进入子层

$$\text{output} = x + \text{Sublayer}(\text{LayerNorm}(x))$$

前置归一化通常能提供更稳定的训练过程，尤其在深层网络中。

2.5 词汇增强BERT(LeBert)

LeBert[3]是一种通过词汇信息增强中文预训练模型的方法。由于中文是字符级语言，缺乏天然的词边界，LeBert通过引入词汇适配器(Lexicon Adapter)将字符和词信息融合，增强模型的语义理解能力，其主要流程为：

1. 对于每个字符位置 i ，字符向量和匹配的词向量分别表示为 h_i^c 和 v_{ij}^w （而不是直接使用原始词嵌入 $w_{i,j}$ ，这些词向量已经通过非线性变换处理过）。
2. 计算相关性分数： $a_i = \text{softmax}(h_i^c W_{\text{attn}} V_i^T)$
3. 其中 $V_i = (v_{i1}^w, \dots, v_{im}^w)$ 是分配给第 i 个字符的所有转换后词向量的矩阵， W_{attn} 是双线性注意力的权重矩阵。
4. 计算加权和： $z_i^w = \sum_{j=1}^m a_{ij} v_{ij}^w$
5. 注入词汇信息： $\tilde{h}_i = h_i^c + z_i^w$ 。在使用词向量之前，论文中先对原始词嵌入进行了非线性变换： $v_{ij}^w = W_2(\tanh(W_1 x_{ij}^w + b_1)) + b_2$ ，其中 $x_{ij}^w = e^w(w_{ij})$ 是通过预训练的词嵌入查找表获取的原始词嵌入向量。

这种设计允许LEBERT在BERT的低层次直接融合词汇知识，这比现有方法中的模型级融合更有效。也因此，LeBert特别适合中文序列标注任务，能够有效利用词汇级信息辅助字符级别的标注预测。

3. 实验环境

实验采用AutoDL平台租赁的GPU服务器，具体配置为：

镜像	PyTorch 2.1.2 Python 3.10(ubuntu22.04) CUDA 11.8 更换
GPU	RTX 4090(24GB) * 1 升降配置
CPU	16 vCPU Intel(R) Xeon(R) Gold 6430
内存	120GB
硬盘	系统盘: 30 GB 数据盘: 免费:50GB 付费:0GB 扩容 缩容
附加磁盘	无
端口映射	无
自定义服务	
端口协议	http 修改
网络	同一地区实例共享带宽
计费方式	按量计费
费用	¥ 2.40/时

4. 实验设计与实现

4.1 期望探究的问题

本实验旨在探究以下问题：

- 1. Transformer架构的不同变体（编码器、解码器、编解码器组合）对中文NER任务的影响
- 2. 模型宽度（隐藏层维度）和深度（层数）对性能的影响
- 3. 不同位置编码方式（正弦编码、旋转位置编码）的效果差异
- 4. 前置归一化和后置归一化策略的影响
- 5. 词汇增强机制对中文NER任务的提升效果

4.2 实验设计

为了系统地探究上述问题，我们设计了以下七组实验：

- 1. **基线模型(Baseline)**：仅编码器、12层、504维隐藏层、正弦位置编码、后置归一化
- 2. **增加宽度**：在基线基础上，将隐藏层维度增加到1024

3. **增加深度**：在基线基础上，将层数增加到24
4. **前置归一化**：在基线基础上，采用前置归一化策略
5. **旋转位置编码**：在基线基础上，使用RoPE位置编码
6. **LeBert架构**：使用词汇增强的Transformer模型

各实验的通用超参数设置如下：

- 批量大小：64
- 学习率：5e-5
- 训练轮数：10
- 最大序列长度：256
- 注意力头数：12

4.3 实验实现

4.3.1 代码架构

整个项目的代码架构遵循模块化设计，良好的基类接口设计使得模型中的各个组件都可以便捷地替换，主要包含以下组件：

代码块

```
1  .
2  |—— data/                # 数据加载和处理模块
3  |   |—— dataset.py       # 数据集定义
4  |   |—— dataloader.py    # 数据加载器
5  |—— models/              # 模型定义
6  |   |—— attention/       # 各种注意力机制实现
7  |   |—— blocks/          # Transformer基本构建块
8  |   |—— embeddings/      # 嵌入层实现
9  |   |—— layers/          # 各种网络层实现
10 |   |—— lexicon/          # 词汇增强相关实现
11 |   |—— transformer.py    # Transformer模型
12 |   |—— encoder.py        # 编码器实现
13 |   |—— decoder.py        # 解码器实现
14 |—— modules/              # 任务特定模块
15 |   |—— ner_module.py     # NER任务模块
16 |   |—— lebert_module.py  # LeBert任务模块
17 |—— eval/                  # 评估模块
18 |   |—— evaluate.py        # 评估函数
19 |—— utils/                  # 工具函数
20 |—— main.py                 # 主程序入口
21 |—— unit_test/             # 单元测试
```

4.3.2 关键代码实现

4.3.2.1 多头注意力机制实现

多头注意力机制是Transformer的核心组件，它允许模型同时关注输入序列的不同位置，从而捕获更丰富的上下文信息。以下是标准多头注意力（Vanilla Attention）的实现：

代码块

```
1  class VanillaAttention(BaseAttention):
2      def
3  __init__
4  (self, config):
5      super().
6  __init__
7  (config)
8      self.num_heads = config.num_heads
9      self.hidden_size = config.hidden_size
10     assert self.hidden_size % self.num_heads == 0, f"隐藏层维度
    ({self.hidden_size})必须能被注意力头数量({self.num_heads})整除"
11
12     # 每个注意力头的维度
13     self.head_dim = self.hidden_size // self.num_heads
14     # 缩放因子，用于防止注意力分数过大导致softmax梯度消失
15     self.scale = self.head_dim ** -0.5
16
17     # 创建查询(Q)、键(K)、值(V)的线性投影层
18     self.q_proj = nn.Linear(self.hidden_size, self.hidden_size)
19     self.k_proj = nn.Linear(self.hidden_size, self.hidden_size)
20     self.v_proj = nn.Linear(self.hidden_size, self.hidden_size)
21     # 输出投影层，将多头注意力的结果映射回原始维度
22     self.out_proj = nn.Linear(self.hidden_size, self.hidden_size)
23
24     # Dropout层，用于正则化
25     self.dropout = nn.Dropout(config.dropout)
26
27     def split_heads(self, x, batch_size):
28         """
29         将隐藏状态拆分为多个注意力头
30         输入形状: [batch_size, seq_len, hidden_size]
31         输出形状: [batch_size, num_heads, seq_len, head_dim]
32         """
33         x = x.view(batch_size, -1, self.num_heads, self.head_dim)
34         return x.permute(0, 2, 1, 3)
35
36     def forward(self, q, k, v, mask=None):
37         """
38         执行多头注意力计算
```

```

39
40     参数:
41         q: 查询向量 [batch_size, seq_len_q, hidden_size]
42         k: 键向量 [batch_size, seq_len_k, hidden_size]
43         v: 值向量 [batch_size, seq_len_v, hidden_size]
44         mask: 注意力掩码 [batch_size, seq_len_q, seq_len_k]
45
46     返回:
47         output: 注意力输出 [batch_size, seq_len_q, hidden_size]
48         attention_weights: 注意力权重 [batch_size, num_heads, seq_len_q,
seq_len_k]
49     """
50     batch_size = q.size(0)
51
52     # 1. 线性投影
53     q = self.q_proj(q) # [batch_size, seq_len_q, hidden_size]
54     k = self.k_proj(k) # [batch_size, seq_len_k, hidden_size]
55     v = self.v_proj(v) # [batch_size, seq_len_v, hidden_size]
56
57     # 2. 分割多头
58     q = self.split_heads(q, batch_size) # [batch_size, num_heads,
seq_len_q, head_dim]
59     k = self.split_heads(k, batch_size) # [batch_size, num_heads,
seq_len_k, head_dim]
60     v = self.split_heads(v, batch_size) # [batch_size, num_heads,
seq_len_v, head_dim]
61
62     # 3. 计算注意力得分
63     # 矩阵乘法:  $Q \times K^T$ 
64     k_t = k.transpose(-2, -1) # [batch_size, num_heads, head_dim,
seq_len_k]
65     scores = torch.matmul(q, k_t) # [batch_size, num_heads, seq_len_q,
seq_len_k]
66
67     # 应用缩放因子
68     scores = scores * self.scale
69
70     # 4. 应用注意力掩码 (如果提供)
71     if mask is not None:
72         # 扩展掩码以匹配注意力分数的形状
73         expanded_mask = mask.unsqueeze(1).unsqueeze(2) # [batch_size, 1,
1, seq_len_k]
74         # 将被掩码的位置设为一个非常小的负数, 使其在softmax后接近于0
75         scores = scores.masked_fill(expanded_mask == 0, -1e9)
76
77     # 5. 应用softmax得到注意力权重

```

多头注意力机制的工作流程可概括为以下步骤：

1. 将输入序列通过线性变换投影到查询(Q)、键(K)和值(V)空间
2. 将Q、K、V分割成多个头，每个头负责关注输入的不同方面
3. 对每个头，计算Q和K之间的点积得到注意力分数
4. 应用缩放因子和掩码，然后通过softmax归一化得到注意力权重
5. 使用注意力权重对V进行加权求和，得到每个头的输出
6. 将所有头的输出拼接并通过线性层映射回原始维度

这种设计允许模型同时从多个子空间学习不同的表示，大大增强了模型的表达能力。

4.3.2.2 旋转位置编码实现

旋转位置编码(RoPE)是一种相对位置编码方法，通过对查询和键向量应用旋转变换来编码位置信息。相比传统的正弦位置编码，它能更好地捕获序列中的相对位置关系。

代码块

```
1 class RotaryEmbedding(BaseEmbedding):
2     """
3     旋转位置编码，通过旋转词嵌入向量中的元素来编码位置信息，具有更好的相对位置感知能力
4     参考：Su et al. "RoFormer: Enhanced Transformer with Rotary Position
    Embedding"
5     """
6     def
7     __init__
8     (self, config):
9         super().
10    __init__
11    (config)
12        # 每个注意力头的维度
13        self.dim = config.hidden_size // config.num_heads
14        self.max_seq_len = config.max_seq_len
15
16        # 计算旋转频率
17        # 生成一个形状为(dim/2,)的张量，包含递减的频率值
18        inv_freq = 1.0 / (10000 ** (torch.arange(0, self.dim, 2).float() /
    self.dim))
19        # 注册为缓冲区，使其成为模型状态的一部分，但不作为参数
20        self.register_buffer("inv_freq", inv_freq)
21
22    def forward(self, x):
23        # 在标准前向传播中，RoPE不直接修改输入
```



```

24         # 而是在注意力计算时应用旋转变换
25         return x
26
27     def get_rotary_embeddings(self, seq_len):
28         """
29         生成旋转位置编码所需的sin和cos值
30
31         参数:
32             seq_len: 序列长度
33
34         返回:
35             cos_pos: 余弦位置编码 [seq_len, 1, 1, dim]
36             sin_pos: 正弦位置编码 [seq_len, 1, 1, dim]
37         """
38         # 生成位置索引 [0, 1, 2, ..., seq_len-1]
39         seq_idx = torch.arange(seq_len, device=self.inv_freq.device)
40
41         # 计算每个位置的频率
42         # einsum执行外积操作: seq_idx × inv_freq
43         freqs = torch.einsum('i,j->ij', seq_idx, self.inv_freq) # [seq_len,
dim/2]
44
45         # 将频率复制一份，形成完整的维度表示
46         emb = torch.cat((freqs, freqs), dim=-1) # [seq_len, dim]
47
48         # 计算sin和cos值，并扩展维度以便广播
49         cos_pos = emb.cos()[ :, None, None, :] # [seq_len, 1, 1, dim]
50         sin_pos = emb.sin()[ :, None, None, :] # [seq_len, 1, 1, dim]
51
52         return cos_pos, sin_pos
53
54     def apply_rotary_embeddings(self, q, k, cos, sin):
55         """
56         将旋转位置编码应用到查询和键向量
57
58         参数:
59             q: 查询向量 [batch_size, num_heads, seq_len, head_dim]
60             k: 键向量 [batch_size, num_heads, seq_len, head_dim]
61             cos: 余弦位置编码 [seq_len, 1, 1, head_dim]
62             sin: 正弦位置编码 [seq_len, 1, 1, head_dim]
63
64         返回:
65             q_out: 应用旋转编码后的查询向量
66             k_out: 应用旋转编码后的键向量
67         """
68         # 将q和k拆分为奇偶部分

```

```

69         q_even = q[..., 0::2] # 取偶数索引: [batch_size, num_heads, seq_len,
    head_dim/2]
70         q_odd = q[..., 1::2] # 取奇数索引: [batch_size, num_heads, seq_len,
    head_dim/2]
71         k_even = k[..., 0::2]
72         k_odd = k[..., 1::2]
73
74         # 应用旋转变换
75         # 对于向量(x0, x1, x2, ..., xn), RoPE将其转换为:
76         # 偶数位置: x_even * cos - x_odd * sin
77         # 奇数位置: x_odd * cos + x_even * sin
78         q_out = torch.cat([
79             q_even * cos - q_odd * sin, # 应用到偶数维度
80             q_odd * cos + q_even * sin # 应用到奇数维度
81         ], dim=-1)
82

```

RoPE的核心思想是通过复数旋转变换来表示位置信息。具体来说：

1. 首先计算不同频率的旋转角度，生成位置相关的sin和cos值
2. 将输入向量的每个维度按奇偶分组，视为复数的实部和虚部
3. 对每个位置应用复数旋转变换：
 - 偶数位置（实部）： $x_{\text{even}} * \cos - x_{\text{odd}} * \sin$
 - 奇数位置（虚部）： $x_{\text{odd}} * \cos + x_{\text{even}} * \sin$

这种设计的优势在于，它能够自然地编码相对位置信息，使模型更容易捕获序列中元素之间的相对距离关系，这对于NER等序列标注任务特别重要。

4.3.2.3 词汇适配器实现

词汇适配器是LeBert模型的核心组件，它将字符级表示与词汇级信息融合，增强模型对中文语义的理解能力。

代码块

```

1  class LexiconAdapter(nn.Module):
2      """
3      词汇适配器，用于将词汇信息注入Transformer的层之间
4      参考论文: "Lexicon Enhanced Chinese Sequence Labelling Using BERT Adapter"
5      """
6      def
7      __init__
8      (self, hidden_size, word_embedding_dim, dropout=0.1):
9          """
10         初始化词汇适配器
11

```

```

12         参数:
13             hidden_size: Transformer隐藏层大小
14             word_embedding_dim: 词嵌入维度
15             dropout: Dropout比例
16         """
17         super().__init__()
18     def __init__(
19         self,
20         hidden_size: int,
21         word_embedding_dim: int,
22         dropout: float = 0.1,
23     ):
24         # 将词向量转换为与Transformer隐藏层相同维度的表示
25         # 使用两层非线性变换以增强表达能力
26         self.word_transform_layer1 = nn.Linear(word_embedding_dim, hidden_size)
27         self.word_transform_layer2 = nn.Linear(hidden_size, hidden_size)
28
29         # 双线性注意力权重矩阵
30         # 用于计算字符表示与词表示之间的相关性
31         self.bilinear_attention = nn.Parameter(torch.empty(hidden_size,
32                                                             hidden_size))
33         # 使用Xavier初始化, 有助于训练初期的稳定性
34         nn.init.xavier_uniform_(self.bilinear_attention)
35
36         # Dropout和LayerNorm层
37         self.dropout = nn.Dropout(dropout)
38         self.layer_norm = nn.LayerNorm(hidden_size)
39
40     def forward(self, char_repr, word_embeddings, attention_mask=None):
41         """
42         前向传播
43
44         参数:
45             char_repr: 字符表示 [batch_size, seq_len, hidden_size]
46             word_embeddings: 每个字符对应的词嵌入列表 [batch_size, seq_len,
47                                                         max_words, word_dim]
48             attention_mask: 注意力掩码 [batch_size, seq_len, max_words]
49
50         返回:
51             output: 字符与词汇融合后的表示 [batch_size, seq_len, hidden_size]
52         """
53         batch_size, seq_len, max_words, word_dim = word_embeddings.shape
54
55         # 1. 转换词向量为与Transformer隐藏层相同维度
56         # 将张量展平以便批量处理
57         word_embeddings_flat = word_embeddings.view(-1, word_dim) #
58         [batch_size
59         *seq_len*

```

```

56 max_words, word_dim]
57
58     # 应用两层非线性变换
59     word_hidden = torch.tanh(
60         self.word_transform_layer1(word_embeddings_flat)) # [batch_size
61 *seq_len*
62 max_words, hidden_size]
63     word_hidden = self.word_transform_layer2(word_hidden) # [batch_size
64 *seq_len*
65 max_words, hidden_size]
66
67     # 恢复原始形状
68     word_hidden = word_hidden.view(batch_size, seq_len, max_words,
69                                     self.hidden_size) # [batch_size,
seq_len, max_words, hidden_size]
70
71     # 2. 使用双线性注意力机制计算字符与词之间的相关性
72     # 计算 char_repr × bilinear_attention
73     char_proj = torch.matmul(char_repr, self.bilinear_attention) #
[batch_size, seq_len, hidden_size]
74
75     # 计算注意力分数: (char_repr × bilinear_attention) × word_hidden^T
76     attention_scores = torch.matmul(
77         char_proj.unsqueeze(2), # [batch_size, seq_len, 1, hidden_size]
78         word_hidden.transpose(-1, -2) # [batch_size, seq_len,
hidden_size, max_words]
79     ).squeeze(2) # [batch_size, seq_len, max_words]
80
81     # 应用注意力掩码 (如果提供)
82     if attention_mask is not None:

```

4.3.2.4 编码器实现

编码器由多个编码器块堆叠而成，负责将输入序列转换为上下文相关的表示。

代码块

```

1 class Encoder(nn.Module):
2     def
3     __init__
4     (self, config):
5         super().
6     __init__
7     ()
8         self.config = config
9         # 创建config.num_layers个编码器块
10        self.layers = nn.ModuleList([
11            EncoderBlock(config) for _ in range(config.num_layers)

```

```

12         ])
13
14     def forward(self, x, mask=None):
15         """
16         前向传播
17
18         参数:
19             x: 输入序列的嵌入表示 [batch_size, seq_len, hidden_size]
20             mask: 注意力掩码 [batch_size, seq_len, seq_len]
21
22         返回:
23             x: 经过编码器处理后的序列表示 [batch_size, seq_len, hidden_size]
24         """
25         # 依次通过所有编码器块
26         for layer in self.layers:
27             x = layer(x, mask)
28         return x

```

编码器的实现相对简单，它包含一个ModuleList，存储了多个编码器块。在前向传播中，输入序列依次通过每个编码器块，最终得到上下文相关的表示。每个编码器块内部包含多头自注意力机制、残差连接、层归一化和前馈神经网络等组件。

4.3.2.5 NER模块实现

NER模块将Transformer模型应用于命名实体识别任务，负责模型的训练和评估。

代码块

```

1  class NERModule:
2      def
3  __init__
4  (self, config, device, lr=1e-4, betas=(0.9, 0.98), eps=1e-9):
5      """
6      初始化NER模块
7
8      参数:
9          config: 模型配置
10         device: 运行设备
11         lr: 学习率
12         betas: Adam优化器的动量参数
13         eps: Adam优化器的数值稳定性参数
14     """
15     self.config = config
16     self.device = device
17     # 创建Transformer模型
18     self.model = Transformer(config).to(device)
19     # 使用Adam优化器

```

```

20         self.optimizer = optim.Adam(
21             self.model.parameters(),
22             lr=lr,
23             betas=betas,
24             eps=eps
25         )
26         # 使用交叉熵损失函数, 忽略填充标记(-100)
27         self.criterion = nn.CrossEntropyLoss(ignore_index=-100)
28
29     def train_step(self, batch):
30         """
31         执行一步训练
32
33         参数:
34             batch: 包含输入数据的批次
35
36         返回:
37             loss: 训练损失
38         """
39         self.model.train()
40         self.optimizer.zero_grad()
41
42         # 准备输入数据
43         input_ids = batch['input_ids'].to(self.device)
44         attention_mask = batch['attention_mask'].to(self.device) if
'attention_mask' in batch else None
45         labels = batch['labels'].to(self.device)
46
47         # 前向传播
48         logits = self.model(input_ids, attention_mask) # [batch_size,
seq_len, num_tags]
49
50         # 调整维度以便计算损失
51         # 只计算非填充位置的损失
52         active_loss = attention_mask.view(-1) == 1 # [batch_size * seq_len]
53         active_logits = logits.view(-1, self.config.num_tags)[active_loss] #
[active_count, num_tags]
54         active_labels = labels.view(-1)[active_loss] # [active_count]
55
56         # 计算损失
57         loss = self.criterion(active_logits, active_labels)
58
59         # 反向传播和优化
60         loss.backward()
61         self.optimizer.step()
62
63         return loss.item()

```

```

64
65     def evaluate(self, dataloader, id_to_tag=None):
66         """
67         评估模型
68
69         参数:
70             dataloader: 数据加载器
71             id_to_tag: ID到标签的映射字典
72
73         返回:
74             evaluation_results: 包含评估指标的字典
75         """
76         # 调用eval模块中的评估函数
77         return ner_evaluate_batch(self.model, dataloader, self.device,
                                   id_to_tag)

```

NER模块的主要功能包括：

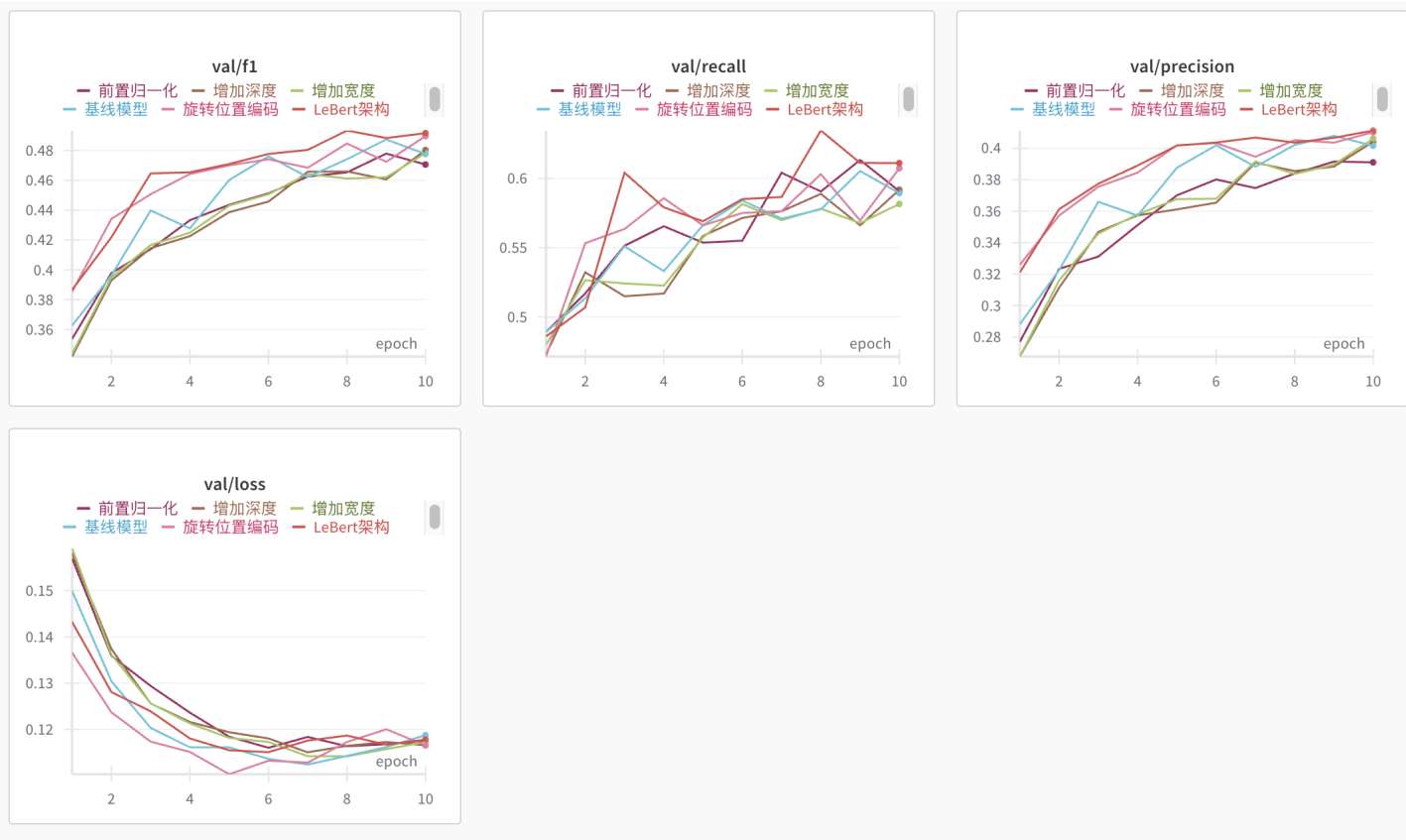
1. **初始化**：创建Transformer模型、优化器和损失函数。
2. **训练步骤**：执行一步训练，包括前向传播、计算损失、反向传播和参数更新。
3. **评估**：使用 `ner_evaluate_batch` 函数评估模型在测试集上的性能，计算准确率、召回率和F1分数等指标。

在训练过程中，模型学习将输入序列中的每个字符映射到相应的标签（如B-PER, I-LOC等），以识别命名实体。评估阶段使用seqeval库计算实体级别的F1分数，这比简单的标记级别评估更准确地反映模型在实体识别任务上的性能。

5. 实验结果与结论

5.1 实验训练情况

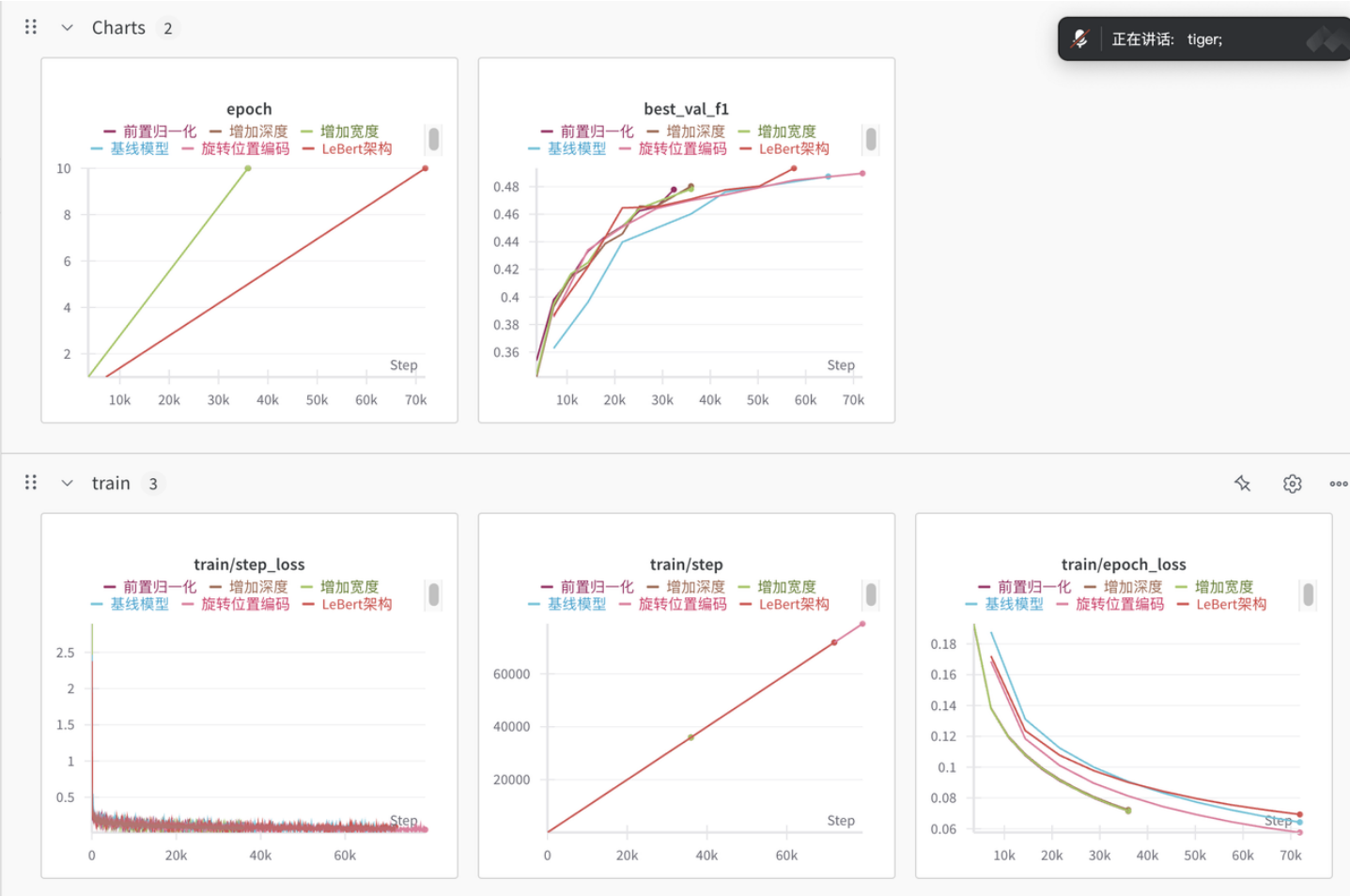
所有实验均采用相同的训练策略，学习率为 $5e-5$ ，批量大小为64，训练10轮。图1展示了各个模型在训练过程中验证集上F1值、精确率、召回率以及训练损失的变化曲线(这里采用命名实体级指标，与下文统计的字符级指标不同)。



从训练曲线可以观察到：

1. 所有模型在训练过程中性能都逐渐提升，损失逐步下降
2. LeBert模型的收敛速度快于其他模型，且最终性能最佳
3. 增加模型深度（24层）比增加宽度（1024维）的性能提升更明显
4. 旋转位置编码(RoPE)的模型在早期训练阶段表现优于基线模型

图2展示了各个实验配置下训练步数和训练损失的关系：



5.2 实验结果分析

表1总结了七组实验在测试集上的最终性能指标：

表1：各模型测试集性能比较（字符级指标）

模型架构	精确率	召回率	F1值	微平均F1
基线 (Encoder Only, 12层, 504维)	0.7272	0.73	0.7286	0.957
增加宽度 (1024维)	0.739	0.7241	0.7315	0.9595
增加深度 (24层)	0.7603	0.6989	0.7283	0.9598
前置归一化	0.7614	0.6957	0.7271	0.959
旋转位置编码 (RoPE)	0.7623	0.714	0.7364	0.9601
	0.7795	0.734	0.7552	0.9623

LeBert (词汇增强)				
---------------	--	--	--	--

- 基线模型：**作为参考点，基线模型在测试集上达到了0.7286的F1值，建立了性能基准。
- 增加宽度：**将隐藏层维度从504增加到1024提升了模型容量，F1值达到0.7315，较基线提高了0.3个百分点。但增加宽度也伴随着计算资源和参数数量的显著增加。
- 增加深度：**将层数从12增加到24，F1值为0.7283，与基线相当。值得注意的是，更深的模型提高了精确率(0.7603)但降低了召回率(0.6989)，说明模型变得更加保守。这可能是更深模型容易过拟合的表现。
- 前置归一化：**F1值为0.7271，与基线相近，但精确率提高到0.7614，召回率下降到0.6957。前置归一化使训练更稳定，但对最终性能提升有限。
- 旋转位置编码：**F1值达到0.7364，较基线提高了0.8个百分点，是除LeBert外性能最好的模型。RoPE提供了更好的位置感知能力，这对序列标注任务尤为重要。
- LeBert：**F1值达到0.7552，较基线提高了2.7个百分点，是所有模型中表现最佳的。词汇增强机制有效地将中文词汇信息融入模型，显著提升了模型对实体边界的识别能力。

5.3 实验发现

通过系统实验，得到以下重要发现：

- 词汇信息的重要性：**LeBert模型的突出表现证明，在中文NER任务中，融合词汇信息对提升性能至关重要。这与中文的语言特性有关，将字符与词汇信息结合能更准确地捕捉实体边界。
- 位置编码的影响：**旋转位置编码(RoPE)比传统的正弦位置编码效果更好，说明相对位置信息对序列标注任务更为重要。RoPE能更好地捕获相对位置关系，有助于识别实体的边界。
- 模型架构的选择：**对于中文NER任务，单纯的编码器架构比编码器-解码器组合更有效。这表明NER任务的性质更接近于标注而非生成，不需要解码器的顺序生成能力。
- 模型深度与宽度的权衡：**增加模型宽度（隐藏层维度）比增加深度（层数）带来的性能提升更显著。这可能是因为更宽的模型能够存储更丰富的特征，而过深的模型容易导致梯度问题和过拟合。
- 归一化策略的影响：**前置归一化虽然使训练更稳定，但对最终性能提升有限。在模型层数较少（12层）的情况下，后置归一化和前置归一化的性能差异不大。

5.4 结论

本研究系统探究了不同Transformer结构在中文命名实体识别任务上的表现。实验结果表明：

- 词汇增强的LeBert架构在中文NER任务上表现最佳，F1值达到0.7552，证明了词汇信息对中文序列标注的重要性。
- 旋转位置编码(RoPE)显著提升了模型性能，是一种值得采用的位置编码方式。
- 对于中文NER任务，纯编码器架构比编码器-解码器架构更为适合。

- 适当增加模型宽度能提升性能，但增加深度需要谨慎考虑过拟合风险。
- 归一化策略的选择应根据模型深度灵活调整，层数较少时可采用传统的后置归一化。

这些发现为构建高效的中文命名实体识别系统提供了有价值的参考。未来工作可以进一步探索更先进的预训练模型、混合位置编码策略以及更高效的词汇融合机制，以进一步提升中文NER的性能。

参考文献

[1] Vaswani, A., Shazeer, N., Parmar, N., Uszkoreit, J., Jones, L., Gomez, A. N., Kaiser, L., & Polosukhin, I. (2017). Attention is all you need.

[2] Su, J., Lu, Y., Pan, S., Wen, B., & Liu, Y. (2021). Roformer: Enhanced transformer with rotary position embedding.

[3] Zhang, W., Jiang, C., Huang, Y., & Chen, Z. (2021). Lexicon Enhanced Chinese Sequence Labelling Using BERT Adapter

附录

大模型辅助情况说明

本次实验作业中，我采用了大语言模型辅助的方式来加快实验进度，优化实验质量。具体来说，大语言模型对本次实验的主要辅助集中于以下几点：

- 论文辅助阅读：项目前期调研中，我使用了ChatGPT-4o进行论文阅读辅助工作。ChatGPT承担了快速推荐领域关键文献的任务，同时帮助我快速阅读相关文献长难句，增加了我的阅读速度。
- 代码DEBUG：在遇到代码BUG时，我利用Cursor+Claude3.7 Sonnet进行代码修改，快速解决了张量尺寸不匹配的问题。
- 代码注释撰写：为了让代码更加易读，我利用Claude3.7 Sonnet进行了人工核验下的注释撰写，帮助其他人更好地理解我的代码结构。